```
"""
pCloud MCP Server
=================
FastMCP server for the pCloud cloud storage API.
Supports US (api.pcloud.com) and EU (eapi.pcloud.com) data regions.

Auth: Set env var PCLOUD_ACCESS_TOKEN (OAuth2 bearer token).
      Optionally set PCLOUD_REGION=eu to use EU endpoint.
"""

import os
import json
import httpx
from typing import Optional, List, Dict, Any
from pydantic import BaseModel, Field, ConfigDict
from mcp.server.fastmcp import FastMCP

# ── Constants ─────────────────────────────────────────────────────────────
PCLOUD_REGION = os.environ.get("PCLOUD_REGION", "us").lower()
BASE_URL = "https://eapi.pcloud.com" if PCLOUD_REGION == "eu" else "https://api.p
ACCESS_TOKEN = os.environ.get("PCLOUD_ACCESS_TOKEN", "")
TIMEOUT = 60.0

mcp = FastMCP("pcloud_mcp")


# ── Shared HTTP client ────────────────────────────────────────────────────
def _headers() -> Dict[str, str]:
    if not ACCESS_TOKEN:
        raise RuntimeError("PCLOUD_ACCESS_TOKEN environment variable is not set."
    return {"Authorization": f"Bearer {ACCESS_TOKEN}"}


async def _get(path: str, params: Optional[Dict] = None) -> Dict[str, Any]:
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        r = await client.get(f"{BASE_URL}/{path}", headers=_headers(), params=par
        r.raise_for_status()
        data = r.json()
        if data.get("result", 0) != 0:
            raise RuntimeError(f"pCloud API error {data.get('result')}: {data.get
        return data


async def _post(path: str, data: Optional[Dict] = None,
```

```
                files: Optional[Dict] = None, params: Optional[Dict] = None) -> D
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        if files:
            r = await client.post(f"{BASE_URL}/{path}", headers=_headers(),
                                  data=data or {}, files=files, params=params or
        else:
            r = await client.post(f"{BASE_URL}/{path}", headers=_headers(),
                                  data=data or {}, params=params or {})
        r.raise_for_status()
        result = r.json()
        if result.get("result", 0) != 0:
            raise RuntimeError(f"pCloud API error {result.get('result')}: {result
        return result


def _fmt_size(b: int) -> str:
    for unit in ["B", "KB", "MB", "GB", "TB"]:
        if b < 1024:
            return f"{b:.1f} {unit}"
        b /= 1024
    return f"{b:.1f} PB"


def _fmt_entry(e: Dict) -> str:
    icon = "📁" if e.get("isfolder") else "📄"
    size = f"  {_fmt_size(e.get('size', 0))}" if not e.get("isfolder") else ""
    return f"{icon} {e.get('name')}  (id={e.get('id') or e.get('folderid') or e.g


# ── Input Models ──────────────────────────────────────────────────────
class FolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_id: int = Field(default=0, description="pCloud folder ID (0 = root)")
    recursive: bool = Field(default=False, description="List subfolders recursive


class FileIdInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="pCloud numeric file ID")


class CreateFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    name: str = Field(..., description="New folder name", min_length=1, max_lengt
    parent_folder_id: int = Field(default=0, description="Parent folder ID (0 = r
```

```
class RenameInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="File ID to rename")
    new_name: str = Field(..., description="New filename including extension")


class MoveInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="File ID to move")
    dest_folder_id: int = Field(..., description="Destination folder ID")


class CopyInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="File ID to copy")
    dest_folder_id: int = Field(..., description="Destination folder ID")


class SearchInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    query: str = Field(..., description="Search term", min_length=1, max_length=2
    folder_id: int = Field(default=0, description="Folder to search within (0 = e


class PublicLinkInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="File ID to create public link for")
    expire_days: Optional[int] = Field(default=None, description="Days until link


class UploadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    local_path: str = Field(..., description="Absolute local path to file to uplo
    folder_id: int = Field(default=0, description="pCloud destination folder ID")
    rename_to: Optional[str] = Field(default=None, description="Rename file on up


class DownloadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: int = Field(..., description="pCloud file ID to download")
    local_path: str = Field(..., description="Absolute local destination path")


class DeleteFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_id: int = Field(..., description="Folder ID to delete")
    recursive: bool = Field(default=False, description="Delete folder and all con
```

```
# ── Tools ──────────────────────────────────────────────────────────────
@mcp.tool(name="pcloud_get_user_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def pcloud_get_user_info() -> str:
    """Return pCloud account information: quota, used space, plan, and email."""
    data = await _get("userinfo")
    u = data
    lines = [
        "## pCloud Account Info",
        f"- **Email**: {u.get('email')}",
        f"- **Plan**: {u.get('subscriptionplan', 'Free')}",
        f"- **Total quota**: {_fmt_size(u.get('quota', 0))}",
        f"- **Used**: {_fmt_size(u.get('usedquota', 0))}",
        f"- **Free**: {_fmt_size(u.get('quota', 0) - u.get('usedquota', 0))}",
        f"- **Region**: {PCLOUD_REGION.upper()}",
    ]
    return "\n".join(lines)


@mcp.tool(name="pcloud_list_folder",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def pcloud_list_folder(params: FolderInput) -> str:
    """List contents of a pCloud folder by folder ID.

    Args:
        params: FolderInput with folder_id (0=root) and optional recursive flag.

    Returns:
        Formatted list of files and subfolders with IDs and sizes.
    """
    try:
        p: Dict[str, Any] = {"folderid": params.folder_id, "noshares": 1}
        if params.recursive:
            p["recursive"] = 1
        data = await _get("listfolder", params=p)
        meta = data.get("metadata", {})
        contents = meta.get("contents", [])
        if not contents:
            return f"Folder {params.folder_id} is empty."
        lines = [f"## Folder: {meta.get('name', 'root')}  (id={params.folder_id})
                 f"**{len(contents)} items**", ""]
        folders = [e for e in contents if e.get("isfolder")]
        files   = [e for e in contents if not e.get("isfolder")]
        if folders:
            lines.append("### Folders")
```

```
            lines.extend(_fmt_entry(e) for e in folders)
        if files:
            lines.append("\n### Files")
            lines.extend(_fmt_entry(e) for e in files)
        return "\n".join(lines)
    except Exception as e:
        return f"Error listing folder: {e}"


@mcp.tool(name="pcloud_get_file_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def pcloud_get_file_info(params: FileIdInput) -> str:
    """Get metadata for a pCloud file by its numeric ID."""
    try:
        data = await _get("stat", params={"fileid": params.file_id})
        m = data.get("metadata", {})
        lines = [
            "## File Info",
            f"- **Name**: {m.get('name')}",
            f"- **ID**: {m.get('fileid')}",
            f"- **Size**: {_fmt_size(m.get('size', 0))}",
            f"- **Content-Type**: {m.get('contenttype', 'unknown')}",
            f"- **Created**: {m.get('created')}",
            f"- **Modified**: {m.get('modified')}",
            f"- **Hash**: {m.get('hash')}",
        ]
        return "\n".join(lines)
    except Exception as e:
        return f"Error getting file info: {e}"


@mcp.tool(name="pcloud_create_folder",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_create_folder(params: CreateFolderInput) -> str:
    """Create a new folder inside a parent pCloud folder."""
    try:
        data = await _post("createfolder",
                           data={"name": params.name, "folderid": params.parent_f
        m = data.get("metadata", {})
        return f"✅ Folder created: **{m.get('name')}**  (id={m.get('folderid')})
    except Exception as e:
        return f"Error creating folder: {e}"


@mcp.tool(name="pcloud_delete_file",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def pcloud_delete_file(params: FileIdInput) -> str:
```

```
    """Permanently delete a file from pCloud by file ID."""
    try:
        data = await _post("deletefile", data={"fileid": params.file_id})
        m = data.get("metadata", {})
        return f"🗑️ Deleted: **{m.get('name')}** (id={params.file_id})"
    except Exception as e:
        return f"Error deleting file: {e}"


@mcp.tool(name="pcloud_delete_folder",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def pcloud_delete_folder(params: DeleteFolderInput) -> str:
    """Delete a pCloud folder. Use recursive=True to delete non-empty folders."""
    try:
        endpoint = "deletefolderrecursive" if params.recursive else "deletefolder
        data = await _post(endpoint, data={"folderid": params.folder_id})
        m = data.get("metadata", {})
        return f"🗑️ Folder deleted: **{m.get('name', params.folder_id)}**"
    except Exception as e:
        return f"Error deleting folder: {e}"


@mcp.tool(name="pcloud_rename_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_rename_file(params: RenameInput) -> str:
    """Rename a pCloud file."""
    try:
        data = await _post("renamefile",
                           data={"fileid": params.file_id, "toname": params.new_n
        m = data.get("metadata", {})
        return f"✏️ Renamed to: **{m.get('name')}**"
    except Exception as e:
        return f"Error renaming file: {e}"


@mcp.tool(name="pcloud_move_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_move_file(params: MoveInput) -> str:
    """Move a pCloud file to a different folder."""
    try:
        data = await _post("renamefile",
                           data={"fileid": params.file_id, "tofolderid": params.d
        m = data.get("metadata", {})
        return f"📦 Moved **{m.get('name')}** to folder {params.dest_folder_id}"
    except Exception as e:
        return f"Error moving file: {e}"
```

```
@mcp.tool(name="pcloud_copy_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_copy_file(params: CopyInput) -> str:
    """Copy a pCloud file to another folder."""
    try:
        data = await _post("copyfile",
                           data={"fileid": params.file_id, "tofolderid": params.d
        m = data.get("metadata", {})
        return f"📋 Copied to: **{m.get('name')}** (new id={m.get('fileid')})"
    except Exception as e:
        return f"Error copying file: {e}"


@mcp.tool(name="pcloud_search_files",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def pcloud_search_files(params: SearchInput) -> str:
    """Search for files and folders in pCloud by name."""
    try:
        p: Dict[str, Any] = {"query": params.query, "folderid": params.folder_id}
        data = await _get("searchfiles", params=p)
        items = data.get("items", [])
        if not items:
            return f"No results found for '{params.query}'."
        lines = [f"## Search results for '{params.query}'", f"**{len(items)} foun
        lines.extend(_fmt_entry(e) for e in items[:50])
        if len(items) > 50:
            lines.append(f"... and {len(items) - 50} more")
        return "\n".join(lines)
    except Exception as e:
        return f"Error searching: {e}"


@mcp.tool(name="pcloud_get_public_link",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_get_public_link(params: PublicLinkInput) -> str:
    """Create a public download link for a pCloud file."""
    try:
        p: Dict[str, Any] = {"fileid": params.file_id}
        if params.expire_days:
            p["expire"] = params.expire_days * 86400
        data = await _post("getfilepublink", data=p)
        link = data.get("link", "")
        code = data.get("code", "")
        full = f"https://filedn.com/{code}" if code and not link else link
        return f"🔗 Public link: `{full}`\n- Expires: {'Never' if not params.expi
    except Exception as e:
```

```
        return f"Error creating public link: {e}"


@mcp.tool(name="pcloud_upload_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def pcloud_upload_file(params: UploadInput) -> str:
    """Upload a local file to pCloud. Provide the absolute path on the local mach
    try:
        if not os.path.exists(params.local_path):
            return f"Error: Local file not found: {params.local_path}"
        filename = params.rename_to or os.path.basename(params.local_path)
        with open(params.local_path, "rb") as f:
            content = f.read()
        size = len(content)
        async with httpx.AsyncClient(timeout=300.0) as client:
            r = await client.post(
                f"{BASE_URL}/uploadfile",
                headers=_headers(),
                data={"folderid": params.folder_id, "filename": filename},
                files={"file": (filename, content)},
            )
            r.raise_for_status()
            data = r.json()
        if data.get("result", 0) != 0:
            return f"Upload error: {data.get('error')}"
        fids = data.get("fileids", [])
        return (f"✅ Uploaded **{filename}** ({_fmt_size(size)}) "
                f"to folder {params.folder_id}  (file id={fids[0] if fids else 'u
    except Exception as e:
        return f"Error uploading file: {e}"


@mcp.tool(name="pcloud_download_file",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def pcloud_download_file(params: DownloadInput) -> str:
    """Download a pCloud file to a local path."""
    try:
        # Get download link
        data = await _get("getfilelink", params={"fileid": params.file_id})
        hosts = data.get("hosts", [])
        path  = data.get("path", "")
        if not hosts or not path:
            return "Error: Could not get download link from pCloud."
        url = f"https://{hosts[0]}{path}"
        async with httpx.AsyncClient(timeout=300.0, follow_redirects=True) as cli
            r = await client.get(url)
            r.raise_for_status()
```

```
            os.makedirs(os.path.dirname(params.local_path) or ".", exist_ok=True)
            with open(params.local_path, "wb") as f:
                f.write(r.content)
        return f"✅ Downloaded to {params.local_path}  ({_fmt_size(len(r.content)
    except Exception as e:
        return f"Error downloading file: {e}"


if __name__ == "__main__":
    mcp.run()
```